Task 03: Network Security Monitoring and Analysis

Introduction:

Submitted by: Khush Rupapara
Date: January 24, 2026
Course: VAPT(Intern)

Objective:

To analyze network traffic using Wireshark for passive traffic analysis, Snort for active intrusion detection, and Scapy for generating test traffic. This task also focuses on identifying and responding to common network security threats.

Executive Summary

This report documents hands-on network security tasks including packet sniffing using Scapy, intrusion detection using Snort, and network traffic analysis with Wireshark. The objective was to gain practical experience with essential monitoring and detection tools used by cybersecurity professionals.

All activities were performed on a Kali Linux system within a controlled virtual environment. Key outcomes included capturing and analyzing live network traffic, detecting TCP SYN packets and port scanning activity using custom Snort rules, and validating alerts through generated test traffic.

Overall, the exercise strengthened practical skills in network analysis, intrusion detection, rule-based security monitoring, and understanding real-world attack reconnaissance techniques.

1)Wireshark

Q1. What is Wireshark and why is it used in network security?
Wireshark is a network packet capturing and analysis tool. It is used in network security to monitor live traffic, analyze packets, and identify suspicious activities such as abnormal traffic flow or repeated connection attempts.

Q2. What type of traffic analysis does Wireshark perform?
Wireshark performs passive traffic analysis, which means it only captures and observes network packets without sending any traffic. This makes it safe for monitoring and investigation purposes.

Q3. How can Wireshark help in detecting suspicious network activity?
Wireshark helps detect suspicious activity by using filters to identify patterns like repeated SYN packets, port scanning behavior, unusual traffic spikes, and unexpected ICMP requests.

Process

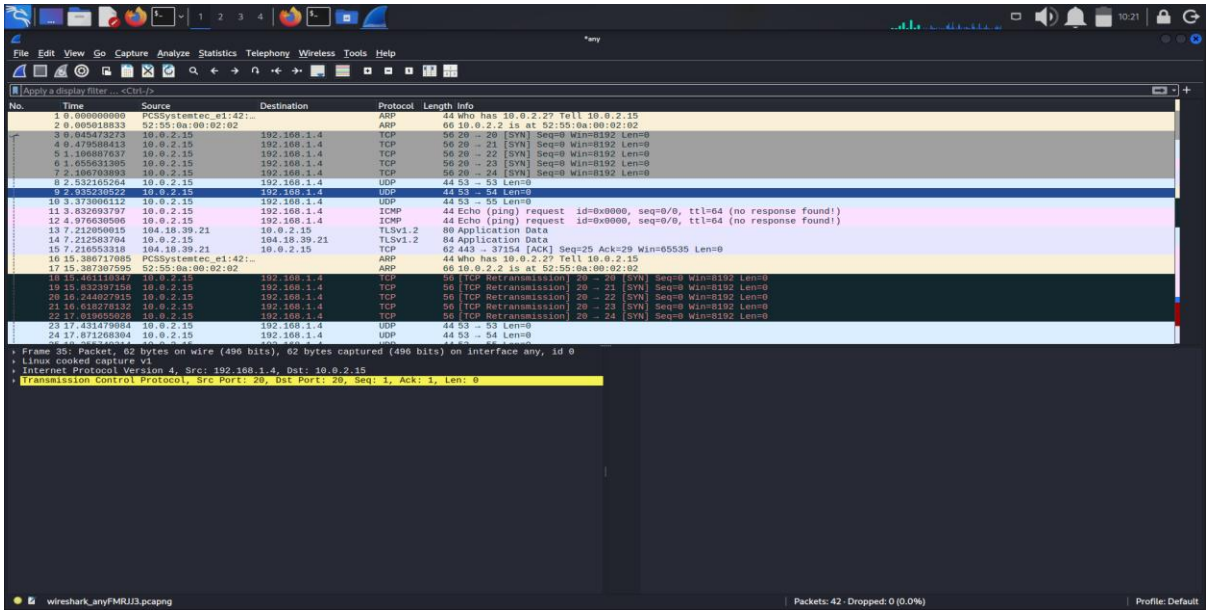
Step 1: Launch Wireshark
First, Wireshark I opened on the Kali Linux system. After launching, the main interface displayed the list of available network interfaces.

Step 2: Select Capture Interface
Then I select any interface for capture traffic from all interfaces. This helps in monitoring complete traffic generated by the system and virtual machines.

Step 3: Generate Network Traffic Using Scapy
To generate test traffic, I create python script using Scapy lib.
This script sent different types of packets such as:

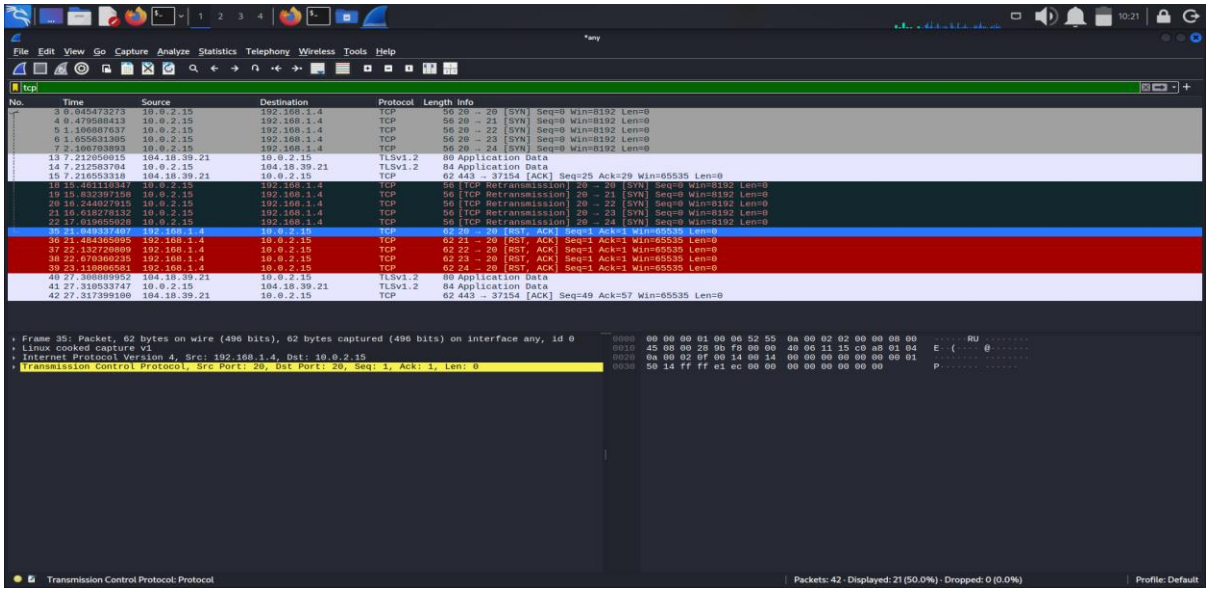
- TCP SYN packets on multiple ports
- UDP packets
- ICMP echo (ping) requests

This traffic was intentionally generated to analyze protocols and detect patterns.



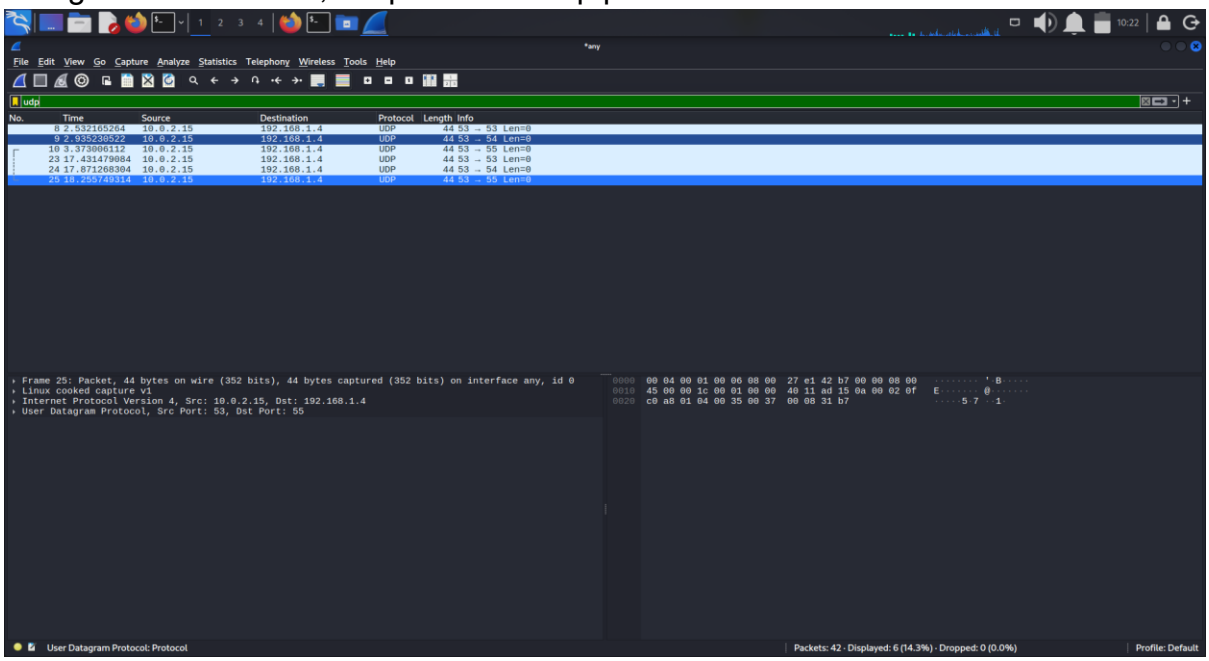
Step 5: Apply TCP Filter

Using Wireshark Filter, I separate the tcp packet from rest of the source.



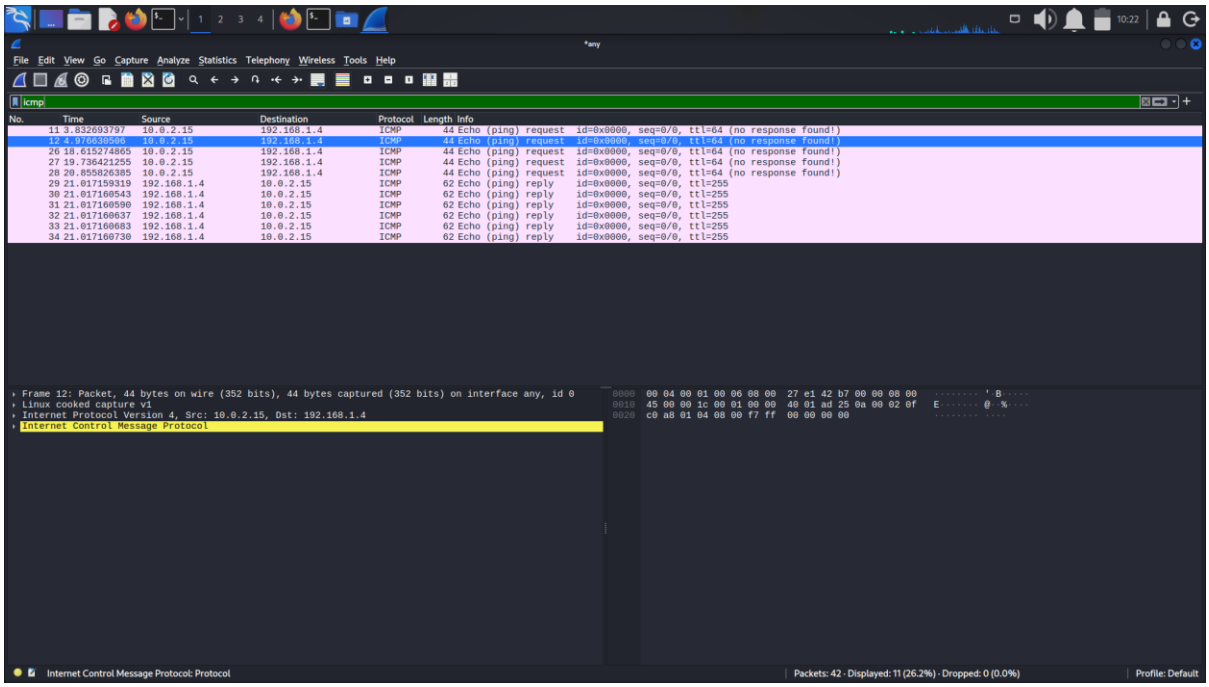
Step 5: Apply UDP Filter

Using Wireshark Filter, I separate the udp packet from rest of the source.



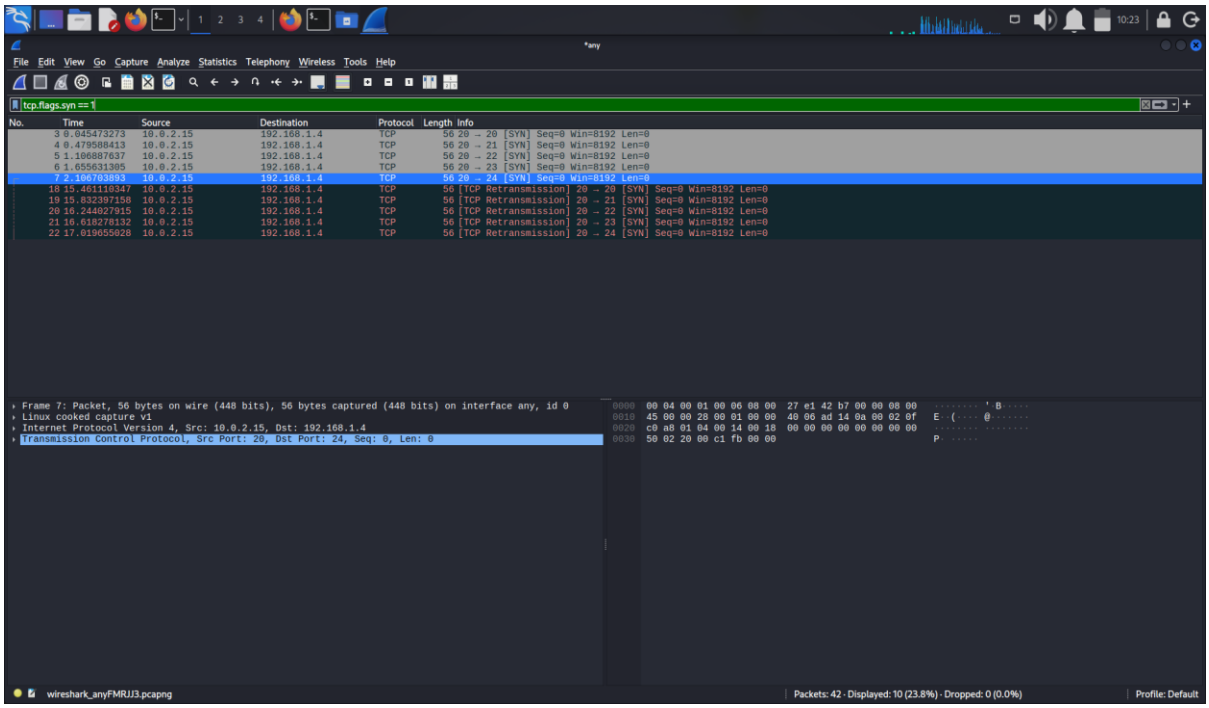
Step 6: Apply ICMP Filter

Using Wireshark Filter, I separate the icmp packet from rest of the source.



Step 5: Apply TCP FLAG Filter

Using Wireshark Filter, I use tcp flag syn filter for separate tcp syn packet from rest of the source.



2)Snort

Q1. What is Snort and why is it used?
Snort is an Intrusion Detection System (IDS) used to monitor network traffic and detect malicious activities such as port scans, SYN floods, and unauthorized access attempts.

Q2. What is IDS mode in Snort?
In IDS mode, Snort analyzes network packets in real time and generates alerts when traffic matches predefined rules. It does not block traffic but helps in identifying potential attacks.

Q3. How does Snort detect network attacks?
Snort detects attacks by using rule-based detection. When network traffic matches a configured rule (such as SYN packets or multiple port requests), Snort raises an alert with details of the suspicious activity.

Process

Step 1: snort install and locate conf file

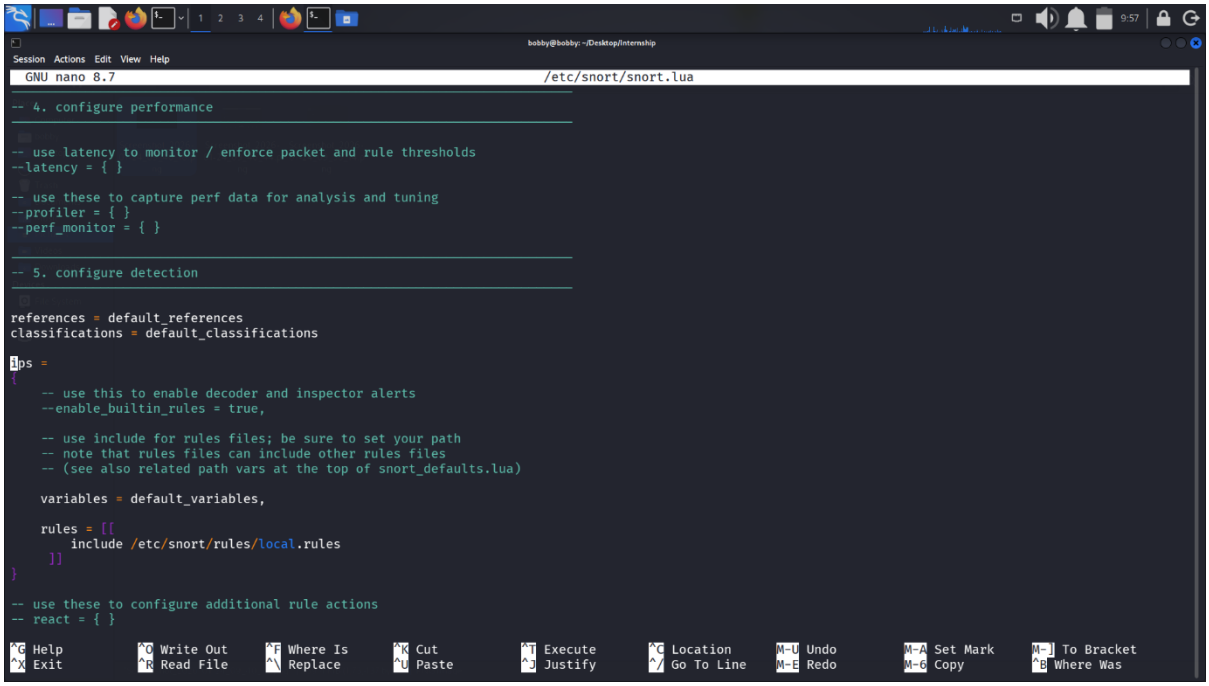
First install snort and then Find conf. file in etc file than add rules.

Step 2: Create Custom Detection Rules

After finding rule file than using nano edit file and add rules. These rules detect TCP SYN packets and potential port scans, helping identify reconnaissance activity, suspicious network behaviour, and early-stage attacks before exploitation occurs**.**

Step 3: Configure Snort Main Configuration File

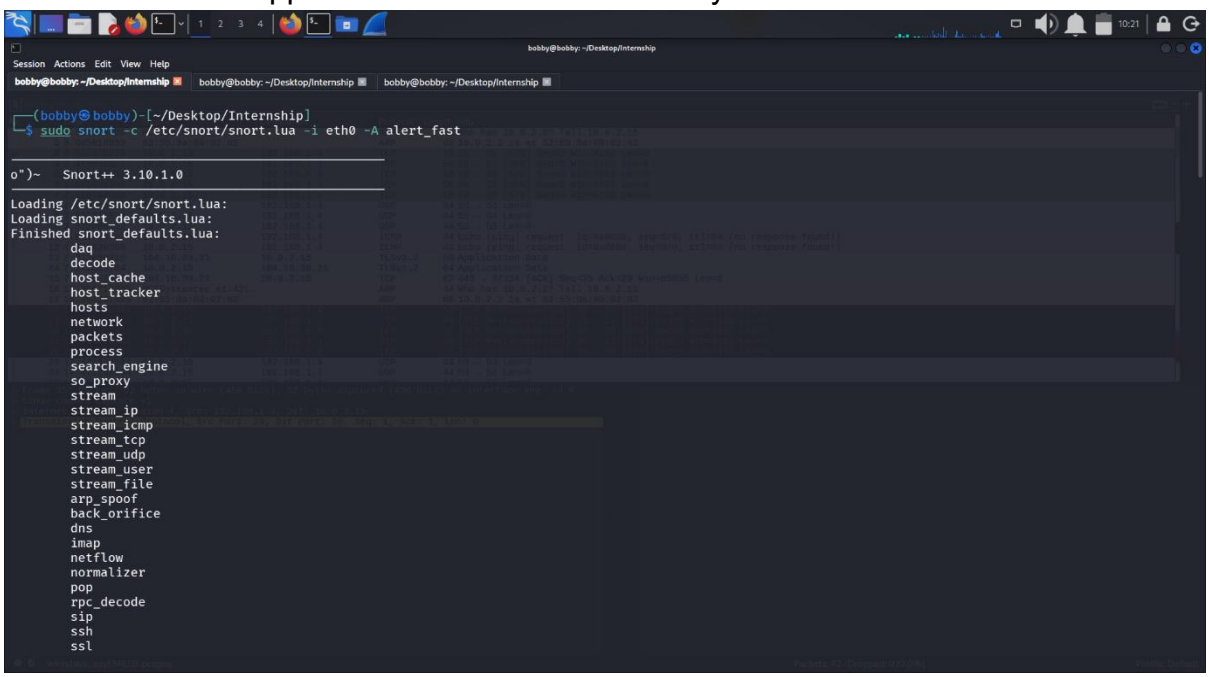
Edit the main Snort configuration file at /etc/snort/snort.lua, That will give path to snort for rules.



Step 4: Start snort scan

using **cmd:**sudo snort -c /etc/snort/snort.lua -I eth0 -A alert_fast

here -c is use for applied rules that I created for my scan.



Step 5: Generate Network Traffic Using Scapy

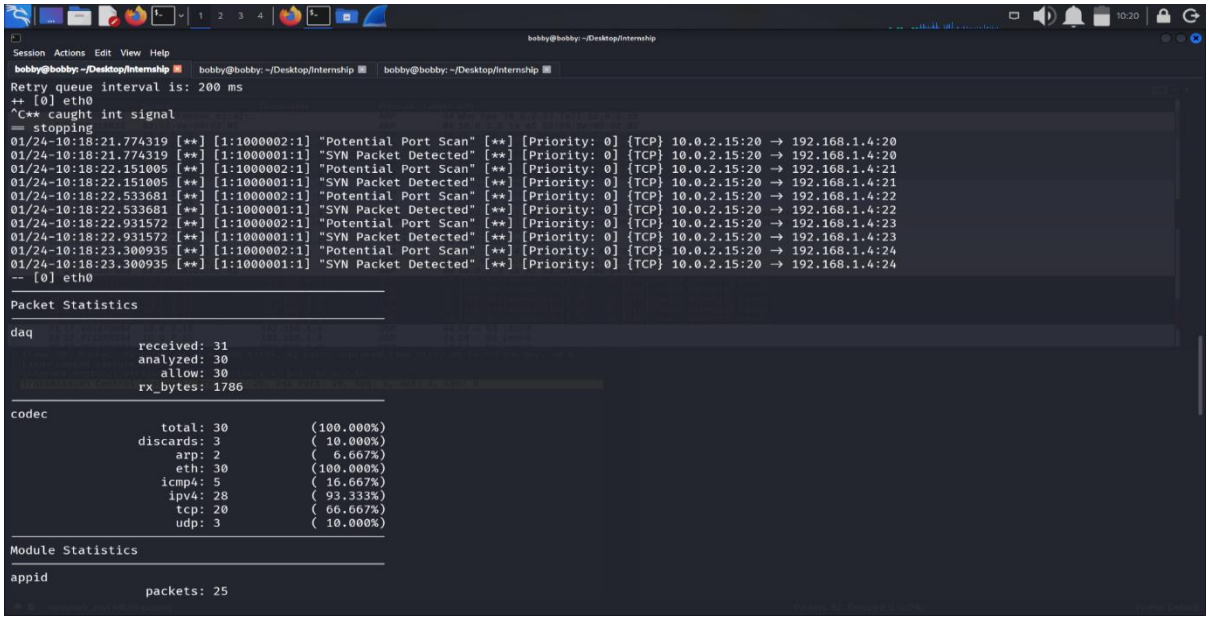
To generate test traffic, I create python script using Scapy lib.

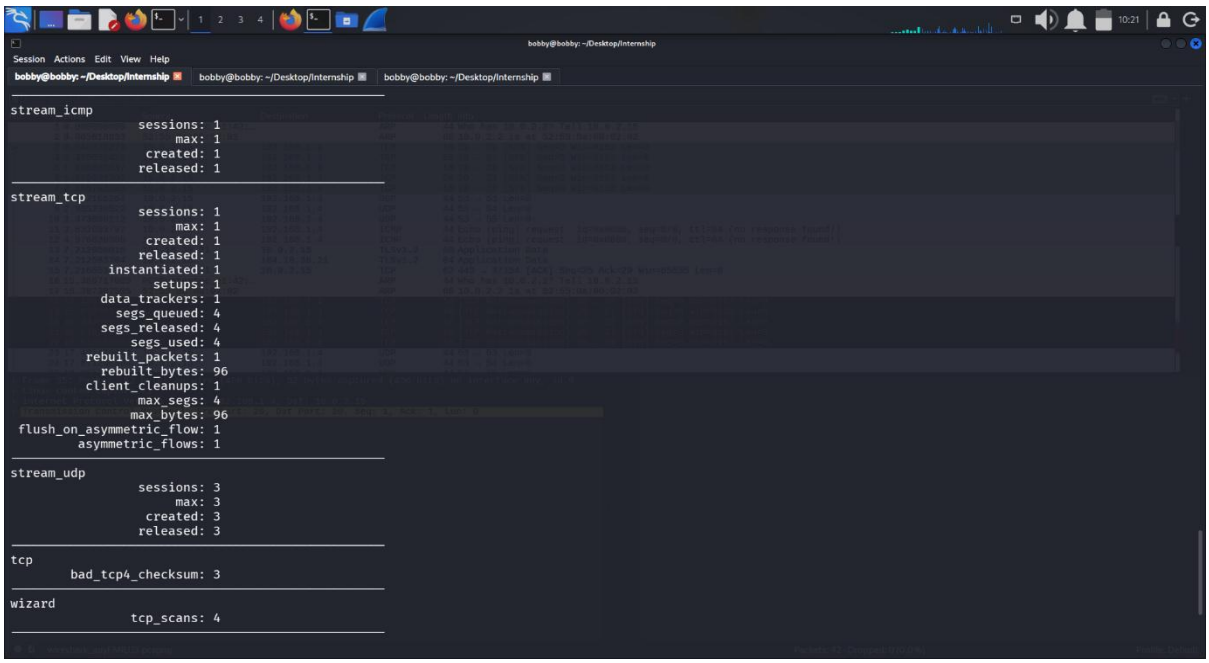
This script sent different types of packets such as:

- TCP SYN packets on multiple ports
- UDP packets
- ICMP echo (ping) requests

This traffic was intentionally generated to analyze protocols and detect patterns.

RESULT OF THIS PROCESS





```
                    ipv4: 28          ( 93.333%)
                     tcp: 20          ( 66.667%)
                     udp: 3           ( 10.000%)
Module Statistics
appid
                packets: 25
        processed_packets: 15
           ignored_packets: 10
            total_sessions: 5
        service_cache_adds: 3
              bytes_in_use: 504
             items_in_use: 3
arp_spoof
                packets: 2
binder
             raw_packets: 12
               new_flows: 5
                inspects: 17
detection
                 analyzed: 30
               hard_evals: 18
                   alerts: 10
             total_alerts: 10
                   logged: 10
ips_actions
                    alert: 10
port_scan
                  packets: 28
                 trackers: 11
search_engine
```

Here is python script(test_packet) output

```
┌──(bobby㉿bobby)-[~/Desktop/Internship]
└─$ sudo python test_packet.py
Starting packet generation for testing IDS...

Sending TCP SYN packets...
TCP SYN sent to port 20
TCP SYN sent to port 21
TCP SYN sent to port 22
TCP SYN sent to port 23
TCP SYN sent to port 24

TCP test completed.

Sending UDP packets...
UDP packet sent to port 53
UDP packet sent to port 54
UDP packet sent to port 55

UDP test completed.

Sending ICMP echo request packets...
ICMP Echo Request sent
ICMP Echo Request sent
ICMP Echo Request sent

All packets sent successfully.

┌──(bobby㉿bobby)-[~/Desktop/Internship]
└─$
```